CS165 Project 1: Analyzing Sorting Algorithms

Will Clark

August 5, 2026

Abstract

This report analyzes the empirical running times of insertion sort, several variations of shell sort, Tim-sort, and skip-list sort with their theoretical complexity. Each algorithm is measured across three input distributions, namely: uniformly distributed, nearly-sorted, and two-alternating. Finally, I declare a best algorithm.

1 Introduction

Sorting algorithms are the *foundation* of computer science. As per Donald Knuth, "virtually every important aspect of programming arises somewhere in the context of sorting or searching!" And not only that, the speed at which we can sort data is often the limiting factor in modern advancements. When discussing the "speed" of an algorithm, a natural crossroads appears: *theoretical* speed, or *practical* speed. This report seeks to derive insights from theoretical explanations, and apply them to empirical results. The canonical choice for analysis is *time vs. input size*, but I'd like to append *comparison count vs. input size* to our analysis. Both cases are plotted on log-log scales to make spotting power-law ($T(n) = cn^k$) running times easier. The below algorithms – and theoretical (worst-case) running times – are listed for reference later.

I) **Insertion Sort**: $\mathcal{O}(N^2)$

II) **Shell Sort** with gap sequence h_k:

- i) **Shell's original** $h_k = \lfloor \frac{N}{2^k} \rfloor,\ k = 1, 2, \ldots, \lfloor \log_2 N \rfloor$: $\mathcal{O}(N^2)$, for $N = 2^p,\ p \in \mathbb{Z}^+$.
- ii) **Frank and Lazarus** $h_k = 2\lfloor \frac{N}{2^{k+1}} \rfloor + 1$, yielding $1, 3, \ldots, 2\lfloor \frac{N}{4} \rfloor + 1$: $\mathcal{O}(N^{3/2})$.
- iii) **A083318** $h_k = 2^k + 1$, with 1, yielding $1, 3, 5, 9, 17, \ldots$: $\mathcal{O}(N^{3/2})$.
- iv) **A003586** $2^p 3^q$ (3-smooth), yielding $1, 2, 3, 4, \ldots$: $\mathcal{O}(N \log^2 N)$.
- v) **A003462** $h_k = \frac{3^k - 1}{2} \leq \lceil \frac{N}{3} \rceil$, yielding $1, 4, 13, 40 \ldots$: $\mathcal{O}(N^{3/2})$.

III) **Tim Sort**: $\mathcal{O}(N \log N)$ worst case, $\mathcal{O}(N)$ best case (adaptive, stable).

IV) **Skip List Sort**: $\mathcal{O}(N \log N)$ expected, $\mathcal{O}(N^2)$ worst case.

Figure 1: Asymptotic complexity of Sorting Algorithms

2 Setup

2.1 Input Distributions

A `Permutation` class was created to handle all input distributions. Each `Permutation` instance stores a `random.Random(SEED)` into `self._rng`. `Permutation` has member functions `uniform`, `two_alt`, and `almost_sorted`, yielding a uniformly distributed sequence, almost sorted sequence (bounded by $\log_2(|S|)$ swaps), and a two-alternating sequence $(1, 3, 5, \ldots, n-1, 2, 4, 6, \ldots, n)$, respectively. Since Python implements the Mersenne Twister for its random number generation, we don't have to worry about any LCG-adjacent issues.

2.2 Data Collection

I used `Pydantic` to store comparisons and time per iteration for every algorithm into `JSON`. Pydantic gave me more flexibility than the suggested `csv` file in the notes. I tested each of the algorithms for 30 trials, on input sizes $16, 32, 64, 128..., 16384$ as to align them with a $\log_2$ axis when graphing.

2.3 Graphing

Using the `JSON` data, I generated log-log plots for *Comparisons vs. Input Size*, and *Time(s) vs. Input Size* using `Matplotlib.pyplot`. Each plot has three best fit lines, one for each input distribution. To avoid cluttering, I am only including the *Time(s) vs. Input Size* plots, but will reference the other graphs if needed.

3 Insertion Sort

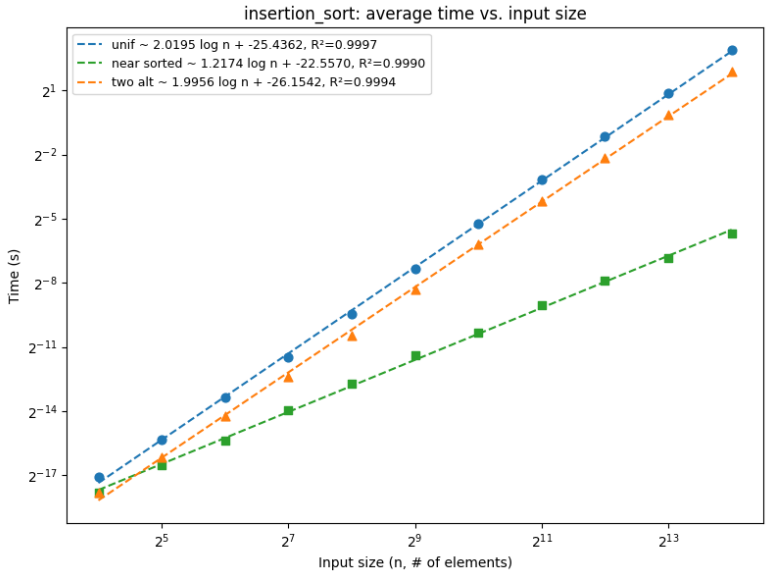
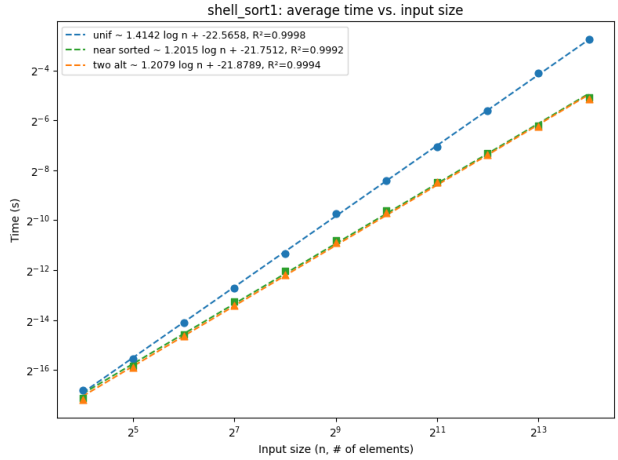


Figure 2: Insertion Sort

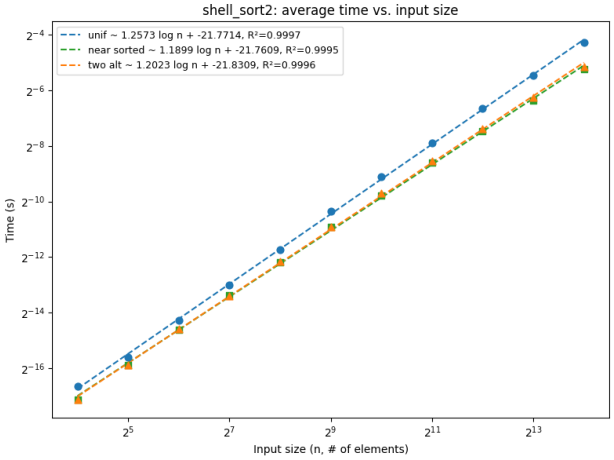
Take c to be the coefficient in front of $\log N$. Insertion sort performed the worst on the uniform distribution ($c = 2.0195$), closely followed by the two-alternating sequence ($c = 1.9956$), then further away by the near-sorted list ($c = 1.2174$). This directly follows from the theory: our empirical running time for uniform and two-alternating is close to $\mathcal{O}(n + k) \subseteq \mathcal{O}(n^2)$, but when the list is almost sorted we have close to linear, $\mathcal{O}(n)$ run-time, since we don't have to "make room" when fixing an inversion, k.

When the input is two-alternating, insertion sort scans halfway up the list (until it reaches S_{n-1}), then has to do $\mathcal{O}(n^2)$ work moving over the odd numbers to insert the even ones. As such, this suggests that insertion sort *is* a viable sort on nearly-sorted sequences, less viable on two-alternating, and should be feared for uniformly distributed inputs.

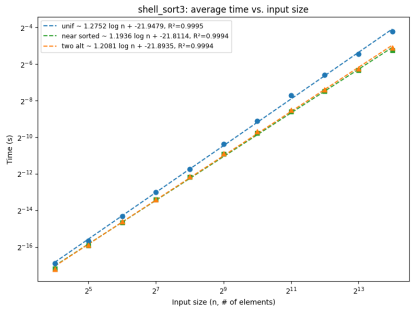
4 Shellsort Variants



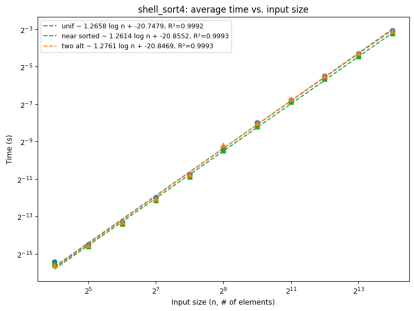
(a) Shell's original



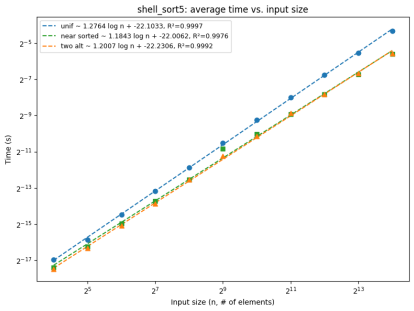
(b) Hibbard



(c) A083318



(d) A003586

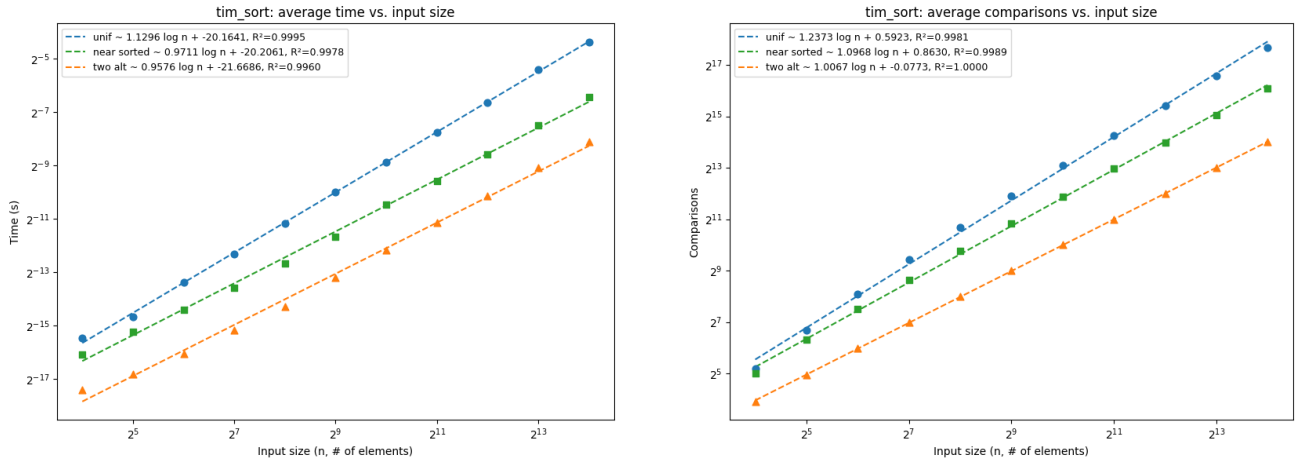


(e) A003462

Figure 3: Log-log runtime plots for the Shellsort gap sequences.

In all of the shell-sort variations, the **nearly-sorted input took the least time**, boasting values of $c = 1.2015, 1.1899, 1.1956, 1.2614, 1.1843$ for each variant, respectively. **A003462** was the **best among all variants for nearly sorted and two-alternating** inputs. **Hibbard** was the **best for uniform** inputs. Here's where things get strange, Shell's original formulation has empirical time complexity for uniform, nearly sorted, and two alternating of $\mathcal{O}_{emp}(n^{1.4102}), \mathcal{O}_{emp}(n^{1.2015}), \mathcal{O}_{emp}(n^{1.2079})$, which is much less than its theoretical $\mathcal{O}(n^2)$ bound. This allows me to conclude that on the average case, Shell-sort is quite an efficient sort. Another observation is that A003586 is almost *input-agnostic*, in the sense that each input produces the same, consistent $\mathcal{O}(n^{\approx 1.26})$ time bound. Since the 3-smooth gaps $\{2^a \cdot 3^b\}$ are dense across all scales, no input permutation can consistently evade early reduction passes, yielding stable $\mathcal{O}(n^{\approx 1.26})$ regardless of input structure.

5 Timsort

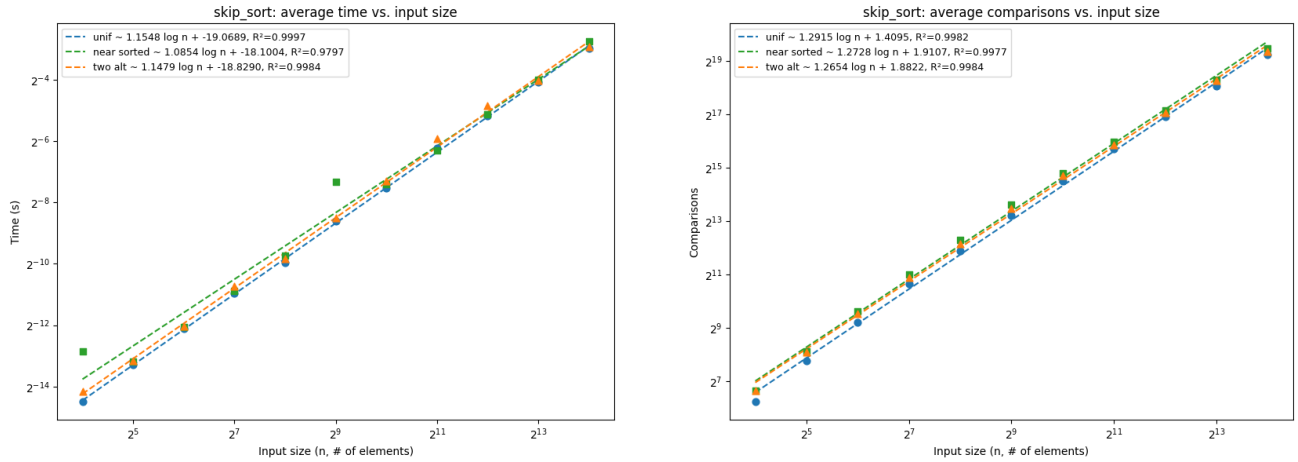


(a) Runtime (b) Comparisons

Figure 4: Tim Sort Plots

The literature tells us that Tim Sort runs in $\mathcal{O}(n + n\mathcal{H}) \subseteq \mathcal{O}(n \log n)$ where $\mathcal{H}$ is the Shannon entropy, but upon a naive glance at the runtime of Tim Sort on the *Time vs. Input Size* plot, we get empirical time complexity of $\mathcal{O}_{emp}(n^{1.1296})$, $\mathcal{O}_{emp}(n^{0.9711})$, and $\mathcal{O}_{emp}(n^{0.9576})$. It's impossible for a sort to be *this* good, so looking at the comparisons plot, we get: $\mathcal{O}_{emp}(n^{1.2373})$, $\mathcal{O}_{emp}(n^{1.0968})$, and $\mathcal{O}_{emp}(n^{1.0067})$. Tim Sort almost runs in $\mathcal{O}(n)$ in the nearly sorted and two-alternating cases. I conjecture that this is because we only have 2 *monotonic non-decreasing* runs in the two-alternating case (the odd half $1, 3, \ldots, n-1$ followed by the even half $2, 4, \ldots, n$), so we are effectively only doing **ONE** merge. It's not quite as extreme in the near-sorted case, but the same logic applies: nearly sorted $\rightarrow$ less runs. On log-log plots, it's quite hard to distinguish between linear and logarithmic functions, but since the exponent, $c \in [1, 1.24]$, it's reasonable to assume an $\mathcal{O}(n \log n)$ average case runtime, but if the input is advantageous, **Tim sort is incredible.**

6 Skip List Sort



(a) Runtime (b) Comparisons

Figure 5: Skip List Sort Plots

Skip sort performs incredibly well on all input distributions, validating its "adaptive-sort" classifi-

cation. The theory tells us that Skip sort has a complexity of $\mathcal{O}(n \log n)$ *with high probability.* It's not deterministic since for each input element we repeat *Bernoulli*($\frac{1}{2}$) until we get 0, which could be infinite if you're (extremely) unlucky. As a general trend, note the $\approx .15$ increase between plots (a) and (b). Since skip sort performs `insert(x)` (and thus `search(x)`) for each element, there is some comparison overhead in actually constructing the skip list that is not reflected in runtime.

Let us now compare skip sort to the rest of the algorithms on each input distribution. For **uniform**, skip sort outperforms every sort except for Tim sort, and by similar reasoning as *Section 5*, we can assume an $\mathcal{O}(n \log n)$ run time, which is strictly less than Tim Sort. For **Almost Sorted**, we only lose to Tim sort in for runtime complexity, but for comparisons we fall middle of the pack. For **Two-Alternating**, we fall right behind Tim Sort but beat everyone else.

7 Input Distribution Sensitivity

To address sensitivity, I split the algorithms into two camps: *input-sensitive* (whose empirical exponent c varies meaningfully across distributions) and *input-agnostic* (whose c is roughly constant). I use the spread $\Delta c = c_{\max} - c_{\min}$ across the three distributions as a rough proxy.

Input-sensitive algorithms. Insertion sort is the most dramatic case, with $\Delta c \approx 0.80$ ($c_{\text{unif}} = 2.02$, $c_{\text{near}} = 1.22$, $c_{\text{two-alt}} = 2.00$). This is expected: insertion sort's running time is governed by the inversion count k, which collapses to $\mathcal{O}(n)$ on nearly-sorted input but blows up to $\Theta(n^2)$ on uniform and two-alternating sequences. Tim sort is also clearly sensitive ($\Delta c \approx 0.23$ in comparisons), since its merge structure exploits existing runs – the two-alternating case degenerates to essentially a single merge, and the nearly-sorted case has few long runs to combine. Skip-list sort shows mild sensitivity ($\Delta c \approx 0.07$ in time), which is consistent with its $\mathcal{O}(n \log n)$ expected-time guarantee being only weakly dependent on input order.

Input-agnostic algorithms. Among the Shellsort variants, A003586 (Figure 3d) is the standout, with all three exponents collapsing to $c \approx 1.26$ ($\Delta c < 0.01$). As argued in Section 4, the density of 3-smooth integers $\{2^a 3^b\}$ across all scales prevents any input permutation from systematically evading early reduction passes. The other Shellsort variants (Shell's original, Hibbard, A083318, A003462) sit in between: their nearly-sorted exponents are noticeably lower (≈ 1.19) than uniform (≈ 1.40 for Shell's original, ≈ 1.26 for the others), but the gap is much smaller than insertion sort's.

In summary, sensitivity tracks the algorithm's ability to exploit existing order. Insertion sort and Tim sort are the most adaptive (and thus most sensitive); A003586 Shellsort is the least adaptive (and thus most agnostic); skip-list sort sits in the middle, gaining little from structure but losing little to the lack of it.

8 Best Algorithm

Tim Sort is the best algorithm we studied, since it adapts to the input distribution's structure and runs incredibly fast. From a philosophical standpoint, there **cannot** be any "best" sorting algorithm, since the context in which you'd use a specific sort is non-static. Also, upon a simple Google search, Powersort replaced Tim Sort in `Python 3.11`, so even in its general domain, Tim-Sort is not the best.